Install Necessary Libraries

In [22]:
```
!pip install tensorflow opencv-python numpy matplotlib scikit-learn adam
```

```
Requirement already satisfied: tensorflow in c:\users\mohan\anaconda3\lib\si
te-packages (2.19.0)
Requirement already satisfied: opencv-python in c:\users\mohan\anaconda3\lib
\site-packages (4.11.0.86)
Requirement already satisfied: numpy in c:\users\mohan\anaconda3\lib\site-pa
ckages (1.26.4)
Requirement already satisfied: matplotlib in c:\users\mohan\anaconda3\lib\si
te-packages (3.9.2)
Requirement already satisfied: scikit-learn in c:\users\mohan\anaconda3\lib
\site-packages (1.5.1)
Requirement already satisfied: adam in c:\users\mohan\anaconda3\lib\site-pac
kages (0.0.0.dev0)
Requirement already satisfied: absl-py>=1.0.0 in c:\users\mohan\anaconda3\li
b\site-packages (from tensorflow) (2.2.2)
Requirement already satisfied: astunparse>=1.6.0 in c:\users\mohan\anaconda3
\lib\site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in c:\users\mohan\anacon
da3\lib\site-packages (from tensorflow) (25.2.10)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in c:\use
rs\mohan\anaconda3\lib\site-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in c:\users\mohan\anacond
a3\lib\site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in c:\users\mohan\anaconda3
\lib\site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in c:\users\mohan\anaconda3
\lib\site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in c:\users\mohan\anaconda3\lib\sit
e-packages (from tensorflow) (24.1)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!
=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in c:\users\mohan\anaconda3\lib\site-pac
kages (from tensorflow) (4.25.3)
Requirement already satisfied: requests<3,>=2.21.0 in c:\users\mohan\anacond
a3\lib\site-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in c:\users\mohan\anaconda3\lib\si
te-packages (from tensorflow) (75.1.0)
Requirement already satisfied: six>=1.12.0 in c:\users\mohan\anaconda3\lib\s
ite-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in c:\users\mohan\anaconda3
\lib\site-packages (from tensorflow) (3.1.0)
Requirement already satisfied: typing-extensions>=3.6.6 in c:\users\mohan\an
aconda3\lib\site-packages (from tensorflow) (4.11.0)
Requirement already satisfied: wrapt>=1.11.0 in c:\users\mohan\anaconda3\lib
\site-packages (from tensorflow) (1.14.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in c:\users\mohan\anacond
a3\lib\site-packages (from tensorflow) (1.71.0)
Requirement already satisfied: tensorboard~=2.19.0 in c:\users\mohan\anacond
a3\lib\site-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in c:\users\mohan\anaconda3\lib
\site-packages (from tensorflow) (3.9.2)
Requirement already satisfied: h5py>=3.11.0 in c:\users\mohan\anaconda3\lib
\site-packages (from tensorflow) (3.11.0)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in c:\users\mohan\ana
conda3\lib\site-packages (from tensorflow) (0.5.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\mohan\anaconda3
\lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\mohan\anaconda3\lib
```

```
\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\mohan\anaconda3
\lib\site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\mohan\anaconda3
\lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: pillow>=8 in c:\users\mohan\anaconda3\lib\sit
e-packages (from matplotlib) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\mohan\anaconda3
\lib\site-packages (from matplotlib) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\mohan\anacon
da3\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.6.0 in c:\users\mohan\anaconda3\lib
\site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\mohan\anaconda3\lib
\site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\mohan\anacon
da3\lib\site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in c:\users\mohan\anaconda
3\lib\site-packages (from astunparse>=1.6.0->tensorflow) (0.44.0)
Requirement already satisfied: rich in c:\users\mohan\anaconda3\lib\site-pac
kages (from keras>=3.5.0->tensorflow) (13.7.1)
Requirement already satisfied: namex in c:\users\mohan\anaconda3\lib\site-pa
ckages (from keras>=3.5.0->tensorflow) (0.0.9)
Requirement already satisfied: optree in c:\users\mohan\anaconda3\lib\site-p
ackages (from keras>=3.5.0->tensorflow) (0.15.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\mohan\an
aconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\mohan\anaconda3\lib
\site-packages (from requests<3,>=2.21.0->tensorflow) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\mohan\anaconda
3\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\mohan\anaconda
3\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (2024.8.30)
Requirement already satisfied: markdown>=2.6.8 in c:\users\mohan\anaconda3\l
ib\site-packages (from tensorboard~=2.19.0->tensorflow) (3.4.1)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in c:\u
sers\mohan\anaconda3\lib\site-packages (from tensorboard~=2.19.0->tensorflo
w) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in c:\users\mohan\anaconda3\l
ib\site-packages (from tensorboard~=2.19.0->tensorflow) (3.0.3)
Requirement already satisfied: MarkupSafe>=2.1.1 in c:\users\mohan\anaconda3
\lib\site-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow)
(2.1.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\mohan\anaco
nda3\lib\site-packages (from rich->keras>=3.5.0->tensorflow) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\mohan\ana
conda3\lib\site-packages (from rich->keras>=3.5.0->tensorflow) (2.15.1)
Requirement already satisfied: mdurl~=0.1 in c:\users\mohan\anaconda3\lib\si
te-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.
1.0)
```

Model Training

- Using MobileNet V2

Importing necessary libraries

In [23]:
```
import os
import random
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dr
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
from sklearn.metrics import precision_score, recall_score, f1_score, classif
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc, roc_auc_score
from itertools import cycle
from PIL import Image
```

Parameters configuration

In [24]:
```
# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
random.seed(42)

# Dataset parameters
train_dir = 'traffic_sign_classification_dataset/train'
test_dir = 'traffic_sign_classification_dataset/test'
num_classes = len(os.listdir(train_dir))  # Number of folders in train direc
batch_size = 32
img_size = (224, 224)  # ImageNet default size
input_shape = (224, 224, 3)

# Disable Tensorflow warnings
tf.get_logger().setLevel('ERROR')
```

Training

In [25]:
```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
```

```
    horizontal_flip=True,
    validation_split=0.2  # Using 20% of training data for validation
)

test_datagen = ImageDataGenerator(rescale=1./255)

# Training data
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset='training'
)

# Validation data
validation_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation'
)

# Get class names (folder names)
class_names = sorted(os.listdir(train_dir))
print(f"Found {len(class_names)} classes: {class_names}")

base_model = MobileNetV2(input_shape=input_shape, include_top=False, weights
base_model.trainable = False  # Freeze base model

model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(num_classes, activation='softmax')  # Output layer
 ])
# Compile the model
model.compile(optimizer=Adam(learning_rate= 0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

checkpoint = ModelCheckpoint('best_model.h5',
                             monitor='val_accuracy',
                             save_best_only=True,
                             mode='max',
                             verbose=1)

early_stopping = EarlyStopping(monitor='val_accuracy',
                               patience=5,
                               restore_best_weights=True)

history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // batch_size,
```

```
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // batch_size,
    epochs=50,
    callbacks=[checkpoint, early_stopping]
)
```

```
Found 3358 images belonging to 55 classes.
Found 812 images belonging to 55 classes.
Found 55 classes: ['Bicycles crossing', 'Children crossing', 'Danger Ahead',
'Dangerous curve to the left', 'Dangerous curve to the right', 'Dont Go Lef
t', 'Dont Go Left or Right', 'Dont Go Right', 'Dont Go straight', 'Dont Go s
traight or Right', 'Dont Go straight or left', 'Dont overtake from Left', 'F
ences', 'Give Way', 'Go Left', 'Go Left or right', 'Go Right', 'Go left or s
traight', 'Go right or straight', 'Go straight', 'Go straight or right', 'He
avy Vehicle Accidents', 'Horn', 'No Car', 'No Uturn', 'No entry', 'No horn',
'No stopping', 'Road Divider', 'Roundabout mandatory', 'Speed limit (15km pe
r h)', 'Speed limit (30km per h)', 'Speed limit (40km per h)', 'Speed limit
(50km per h)', 'Speed limit (5km per h)', 'Speed limit (60km per h)', 'Speed
limit (70km per h)', 'Stop', 'Traffic signals', 'Train Crossing', 'Under Con
struction', 'Unknown1', 'Unknown2', 'Unknown3', 'Unknown4', 'Unknown5', 'Unk
nown7', 'Unknown8', 'Uturn', 'Zebra Crossing', 'ZigZag Curve', 'keep Left',
'keep Right', 'speed limit (80km per h)', 'watch out for cars']
```

```
C:\Users\mohan\anaconda3\Lib\site-packages\keras\src\trainers\data_adapters
\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call
`super().__init__(**kwargs)` in its constructor. `**kwargs` can include `wor
kers`, `use_multiprocessing`, `max_queue_size`. Do not pass these arguments
to `fit()`, as they will be ignored.
  self._warn_if_super_not_called()
```

```
Epoch 1/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.1267 - loss: 3.7605
Epoch 1: val_accuracy improved from -inf to 0.37000, saving model to best_mo
del.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
104/104 ━━━━━━━━━━━━━━━━━━━━ 244s 2s/step - accuracy: 0.1273 - loss: 3.7572
- val_accuracy: 0.3700 - val_loss: 2.7269
Epoch 2/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:33 911ms/step - accuracy: 0.2812 - loss: 2.88
31
```

```
C:\Users\mohan\anaconda3\Lib\site-packages\keras\src\trainers\epoch_iterato
r.py:107: UserWarning: Your input ran out of data; interrupting training. Ma
ke sure that your dataset or generator can generate at least `steps_per_epoc
h * epochs` batches. You may need to use the `.repeat()` function when build
ing your dataset.
  self._interrupted_warning()
```

```
Epoch 2: val_accuracy improved from 0.37000 to 0.38250, saving model to best
_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
104/104 ━━━━━━━━━━━━━━━━━━━━ 46s 437ms/step - accuracy: 0.2812 - loss: 2.883
1 - val_accuracy: 0.3825 - val_loss: 2.7115
Epoch 3/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.3707 - loss: 2.6832
Epoch 3: val_accuracy improved from 0.38250 to 0.56000, saving model to best
_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 230s 2s/step - accuracy: 0.3711 - loss: 2.6813
- val_accuracy: 0.5600 - val_loss: 2.0040
Epoch 4/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:33 909ms/step - accuracy: 0.2812 - loss: 2.95
96
Epoch 4: val_accuracy did not improve from 0.56000
104/104 ━━━━━━━━━━━━━━━━━━━━ 82s 784ms/step - accuracy: 0.2812 - loss: 2.959
6 - val_accuracy: 0.5562 - val_loss: 2.0161
Epoch 5/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.4918 - loss: 2.0695
Epoch 5: val_accuracy improved from 0.56000 to 0.65250, saving model to best
_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 228s 2s/step - accuracy: 0.4921 - loss: 2.0683
- val_accuracy: 0.6525 - val_loss: 1.6036
Epoch 6/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:38 960ms/step - accuracy: 0.5625 - loss: 1.98
71
Epoch 6: val_accuracy did not improve from 0.65250
104/104 ━━━━━━━━━━━━━━━━━━━━ 80s 768ms/step - accuracy: 0.5625 - loss: 1.987
1 - val_accuracy: 0.6488 - val_loss: 1.5965
Epoch 7/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.6071 - loss: 1.6280
Epoch 7: val_accuracy improved from 0.65250 to 0.71875, saving model to best
_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 263s 3s/step - accuracy: 0.6070 - loss: 1.6278
- val_accuracy: 0.7188 - val_loss: 1.3196
Epoch 8/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:38 957ms/step - accuracy: 0.6875 - loss: 1.52
92
Epoch 8: val_accuracy did not improve from 0.71875
104/104 ━━━━━━━━━━━━━━━━━━━━ 81s 782ms/step - accuracy: 0.6875 - loss: 1.529
2 - val_accuracy: 0.7175 - val_loss: 1.3229
Epoch 9/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.6419 - loss: 1.4400
Epoch 9: val_accuracy improved from 0.71875 to 0.74000, saving model to best
_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 225s 2s/step - accuracy: 0.6421 - loss: 1.4395
- val_accuracy: 0.7400 - val_loss: 1.1539
Epoch 10/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:36 941ms/step - accuracy: 0.9062 - loss: 0.72
71
Epoch 10: val_accuracy improved from 0.74000 to 0.74375, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 82s 790ms/step - accuracy: 0.9062 - loss: 0.727
1 - val_accuracy: 0.7437 - val_loss: 1.1370
Epoch 11/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7039 - loss: 1.1754
Epoch 11: val_accuracy improved from 0.74375 to 0.77125, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 262s 3s/step - accuracy: 0.7039 - loss: 1.1755
- val_accuracy: 0.7713 - val_loss: 1.0192
Epoch 12/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:37 951ms/step - accuracy: 0.7500 - loss: 1.03
53
Epoch 12: val_accuracy improved from 0.77125 to 0.78000, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 82s 790ms/step - accuracy: 0.7500 - loss: 1.035
3 - val_accuracy: 0.7800 - val_loss: 1.0023
Epoch 13/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7227 - loss: 1.0882
Epoch 13: val_accuracy improved from 0.78000 to 0.79375, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 229s 2s/step - accuracy: 0.7228 - loss: 1.0879
- val_accuracy: 0.7937 - val_loss: 0.9048
Epoch 14/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:37 945ms/step - accuracy: 0.7500 - loss: 0.78
35
Epoch 14: val_accuracy improved from 0.79375 to 0.80375, saving model to bes
t_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 82s 790ms/step - accuracy: 0.7500 - loss: 0.783
5 - val_accuracy: 0.8037 - val_loss: 0.8665
Epoch 15/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7492 - loss: 0.9817
Epoch 15: val_accuracy improved from 0.80375 to 0.81875, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 226s 2s/step - accuracy: 0.7493 - loss: 0.9814
- val_accuracy: 0.8188 - val_loss: 0.7864
Epoch 16/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:35 928ms/step - accuracy: 0.6562 - loss: 1.10
48
Epoch 16: val_accuracy did not improve from 0.81875
104/104 ━━━━━━━━━━━━━━━━━━━━ 81s 780ms/step - accuracy: 0.6562 - loss: 1.104
8 - val_accuracy: 0.8188 - val_loss: 0.8001
Epoch 17/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7699 - loss: 0.8892
Epoch 17: val_accuracy improved from 0.81875 to 0.82125, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 226s 2s/step - accuracy: 0.7700 - loss: 0.8890
- val_accuracy: 0.8213 - val_loss: 0.7263
Epoch 18/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:34 917ms/step - accuracy: 0.6875 - loss: 1.06
12
Epoch 18: val_accuracy did not improve from 0.82125
104/104 ━━━━━━━━━━━━━━━━━━━━ 83s 797ms/step - accuracy: 0.6875 - loss: 1.061
2 - val_accuracy: 0.8138 - val_loss: 0.7474
Epoch 19/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7888 - loss: 0.8124
Epoch 19: val_accuracy improved from 0.82125 to 0.83500, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 227s 2s/step - accuracy: 0.7888 - loss: 0.8123
- val_accuracy: 0.8350 - val_loss: 0.6903
Epoch 20/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:36 939ms/step - accuracy: 0.6562 - loss: 1.10
99
Epoch 20: val_accuracy improved from 0.83500 to 0.83625, saving model to bes
t_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 83s 793ms/step - accuracy: 0.6562 - loss: 1.109
9 - val_accuracy: 0.8363 - val_loss: 0.6850
Epoch 21/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7955 - loss: 0.7648
Epoch 21: val_accuracy improved from 0.83625 to 0.85000, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 223s 2s/step - accuracy: 0.7955 - loss: 0.7648
- val_accuracy: 0.8500 - val_loss: 0.6163
Epoch 22/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:38 960ms/step - accuracy: 0.9062 - loss: 0.57
09
Epoch 22: val_accuracy did not improve from 0.85000
104/104 ━━━━━━━━━━━━━━━━━━━━ 82s 784ms/step - accuracy: 0.9062 - loss: 0.570
9 - val_accuracy: 0.8425 - val_loss: 0.6252
Epoch 23/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8103 - loss: 0.7077
Epoch 23: val_accuracy improved from 0.85000 to 0.86000, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 224s 2s/step - accuracy: 0.8104 - loss: 0.7075
- val_accuracy: 0.8600 - val_loss: 0.5799
Epoch 24/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:35 930ms/step - accuracy: 0.8438 - loss: 0.64
83
Epoch 24: val_accuracy improved from 0.86000 to 0.86625, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 84s 803ms/step - accuracy: 0.8438 - loss: 0.648
3 - val_accuracy: 0.8662 - val_loss: 0.5743
Epoch 25/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8184 - loss: 0.6456
Epoch 25: val_accuracy improved from 0.86625 to 0.86750, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
104/104 ━━━━━━━━━━━━━━━━━━━━ 227s 2s/step - accuracy: 0.8184 - loss: 0.6456
- val_accuracy: 0.8675 - val_loss: 0.5591
Epoch 26/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:34 920ms/step - accuracy: 0.8125 - loss: 0.50
26
Epoch 26: val_accuracy improved from 0.86750 to 0.87375, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 82s 786ms/step - accuracy: 0.8125 - loss: 0.502
6 - val_accuracy: 0.8737 - val_loss: 0.5570
Epoch 27/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8291 - loss: 0.6346
Epoch 27: val_accuracy improved from 0.87375 to 0.88500, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 225s 2s/step - accuracy: 0.8291 - loss: 0.6344
- val_accuracy: 0.8850 - val_loss: 0.4952
Epoch 28/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:37 949ms/step - accuracy: 0.7812 - loss: 0.69
58
Epoch 28: val_accuracy improved from 0.88500 to 0.88875, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 45s 429ms/step - accuracy: 0.7812 - loss: 0.695
8 - val_accuracy: 0.8888 - val_loss: 0.4978
Epoch 29/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8431 - loss: 0.5865
Epoch 29: val_accuracy improved from 0.88875 to 0.89375, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 262s 3s/step - accuracy: 0.8431 - loss: 0.5863
- val_accuracy: 0.8938 - val_loss: 0.4671
Epoch 30/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:32 900ms/step - accuracy: 0.7188 - loss: 0.94
76
Epoch 30: val_accuracy did not improve from 0.89375
104/104 ━━━━━━━━━━━━━━━━━━━━ 45s 424ms/step - accuracy: 0.7188 - loss: 0.947
6 - val_accuracy: 0.8900 - val_loss: 0.4615
Epoch 31/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8579 - loss: 0.5467
Epoch 31: val_accuracy improved from 0.89375 to 0.90250, saving model to bes
t_model.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 262s 3s/step - accuracy: 0.8578 - loss: 0.5466
- val_accuracy: 0.9025 - val_loss: 0.4234
Epoch 32/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:34 918ms/step - accuracy: 0.8750 - loss: 0.58
67
Epoch 32: val_accuracy did not improve from 0.90250
104/104 ━━━━━━━━━━━━━━━━━━━━ 83s 793ms/step - accuracy: 0.8750 - loss: 0.586
7 - val_accuracy: 0.8988 - val_loss: 0.4521
Epoch 33/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8610 - loss: 0.5270
Epoch 33: val_accuracy did not improve from 0.90250
104/104 ━━━━━━━━━━━━━━━━━━━━ 219s 2s/step - accuracy: 0.8610 - loss: 0.5269
- val_accuracy: 0.8913 - val_loss: 0.4482
Epoch 34/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:35 923ms/step - accuracy: 0.9688 - loss: 0.39
68
Epoch 34: val_accuracy did not improve from 0.90250
104/104 ━━━━━━━━━━━━━━━━━━━━ 83s 795ms/step - accuracy: 0.9688 - loss: 0.396
8 - val_accuracy: 0.9025 - val_loss: 0.4302
Epoch 35/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8763 - loss: 0.4722
Epoch 35: val_accuracy improved from 0.90250 to 0.90500, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 219s 2s/step - accuracy: 0.8763 - loss: 0.4723
- val_accuracy: 0.9050 - val_loss: 0.4103
Epoch 36/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:29 869ms/step - accuracy: 0.8750 - loss: 0.27
47
Epoch 36: val_accuracy improved from 0.90500 to 0.91375, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 44s 421ms/step - accuracy: 0.8750 - loss: 0.274
7 - val_accuracy: 0.9137 - val_loss: 0.3889
Epoch 37/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8817 - loss: 0.4404
Epoch 37: val_accuracy improved from 0.91375 to 0.91875, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
104/104 ━━━━━━━━━━━━━━━━━━━━ 222s 2s/step - accuracy: 0.8817 - loss: 0.4405
- val_accuracy: 0.9187 - val_loss: 0.3767
Epoch 38/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:29 870ms/step - accuracy: 0.8750 - loss: 0.44
92
Epoch 38: val_accuracy did not improve from 0.91875
104/104 ━━━━━━━━━━━━━━━━━━━━ 44s 417ms/step - accuracy: 0.8750 - loss: 0.449
2 - val_accuracy: 0.9162 - val_loss: 0.3614
Epoch 39/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8889 - loss: 0.4360
Epoch 39: val_accuracy improved from 0.91875 to 0.92375, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 263s 3s/step - accuracy: 0.8889 - loss: 0.4360
- val_accuracy: 0.9237 - val_loss: 0.3453
Epoch 40/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:31 891ms/step - accuracy: 0.9062 - loss: 0.40
65
Epoch 40: val_accuracy did not improve from 0.92375
104/104 ━━━━━━━━━━━━━━━━━━━━ 44s 415ms/step - accuracy: 0.9062 - loss: 0.406
5 - val_accuracy: 0.9212 - val_loss: 0.3548
Epoch 41/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8947 - loss: 0.4086
Epoch 41: val_accuracy did not improve from 0.92375
104/104 ━━━━━━━━━━━━━━━━━━━━ 224s 2s/step - accuracy: 0.8947 - loss: 0.4087
- val_accuracy: 0.9137 - val_loss: 0.3503
Epoch 42/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:33 910ms/step - accuracy: 0.8125 - loss: 0.45
10
Epoch 42: val_accuracy did not improve from 0.92375
104/104 ━━━━━━━━━━━━━━━━━━━━ 44s 414ms/step - accuracy: 0.8125 - loss: 0.451
0 - val_accuracy: 0.9125 - val_loss: 0.3500
Epoch 43/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.8897 - loss: 0.4086
Epoch 43: val_accuracy did not improve from 0.92375
104/104 ━━━━━━━━━━━━━━━━━━━━ 223s 2s/step - accuracy: 0.8897 - loss: 0.4085
- val_accuracy: 0.9225 - val_loss: 0.3424
Epoch 44/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:33 909ms/step - accuracy: 0.8750 - loss: 0.28
82
Epoch 44: val_accuracy improved from 0.92375 to 0.92625, saving model to bes
t_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 45s 424ms/step - accuracy: 0.8750 - loss: 0.288
2 - val_accuracy: 0.9262 - val_loss: 0.3287
Epoch 45/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9037 - loss: 0.3634
Epoch 45: val_accuracy improved from 0.92625 to 0.93625, saving model to bes
t_model.h5
```

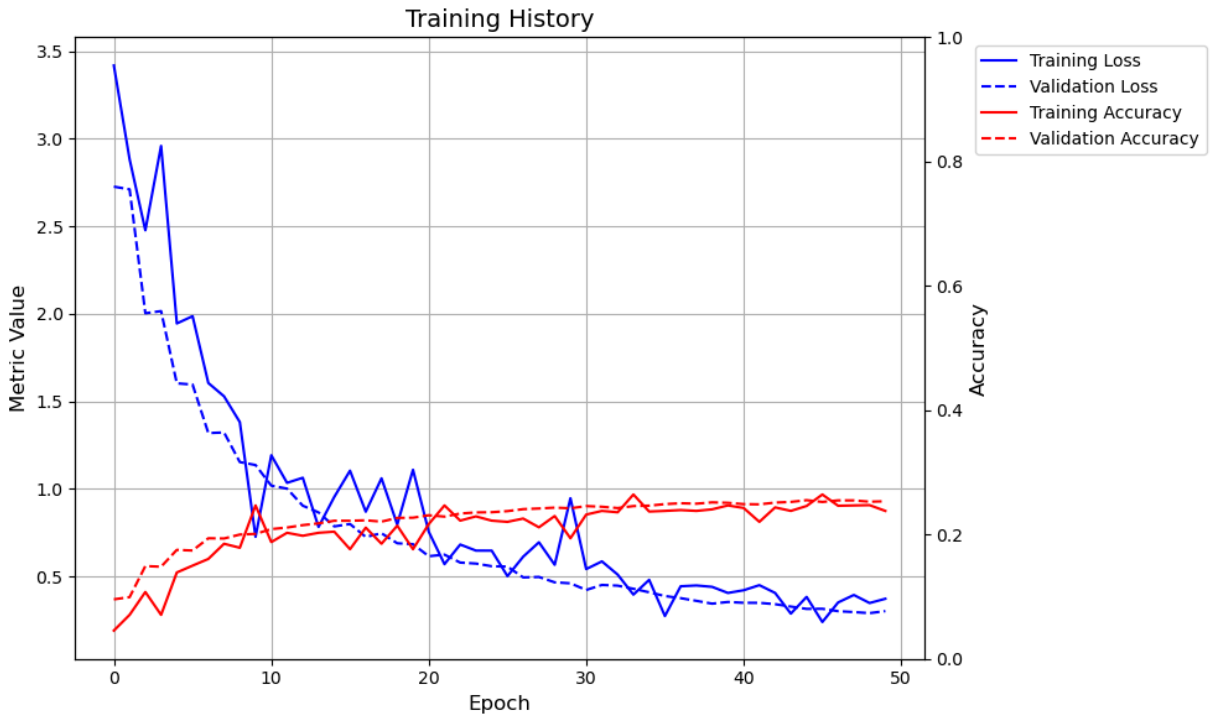
```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.
keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
104/104 ━━━━━━━━━━━━━━━━━━━━ 224s 2s/step - accuracy: 0.9036 - loss: 0.3636
- val_accuracy: 0.9362 - val_loss: 0.3154
Epoch 46/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:27 851ms/step - accuracy: 0.9688 - loss: 0.24
04
Epoch 46: val_accuracy did not improve from 0.93625
104/104 ━━━━━━━━━━━━━━━━━━━━ 44s 423ms/step - accuracy: 0.9688 - loss: 0.240
4 - val_accuracy: 0.9262 - val_loss: 0.3159
Epoch 47/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9083 - loss: 0.3531
Epoch 47: val_accuracy did not improve from 0.93625
104/104 ━━━━━━━━━━━━━━━━━━━━ 264s 3s/step - accuracy: 0.9083 - loss: 0.3531
- val_accuracy: 0.9350 - val_loss: 0.3023
Epoch 48/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:53 1s/step - accuracy: 0.9062 - loss: 0.3949
Epoch 48: val_accuracy did not improve from 0.93625
104/104 ━━━━━━━━━━━━━━━━━━━━ 46s 440ms/step - accuracy: 0.9062 - loss: 0.394
9 - val_accuracy: 0.9350 - val_loss: 0.2972
Epoch 49/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9089 - loss: 0.3395
Epoch 49: val_accuracy did not improve from 0.93625
104/104 ━━━━━━━━━━━━━━━━━━━━ 230s 2s/step - accuracy: 0.9089 - loss: 0.3396
- val_accuracy: 0.9275 - val_loss: 0.2912
Epoch 50/50
  1/104 ━━━━━━━━━━━━━━━━━━━━ 1:39 969ms/step - accuracy: 0.8750 - loss: 0.37
32
Epoch 50: val_accuracy did not improve from 0.93625
104/104 ━━━━━━━━━━━━━━━━━━━━ 46s 435ms/step - accuracy: 0.8750 - loss: 0.373
2 - val_accuracy: 0.9300 - val_loss: 0.3027
```

Plot Graphs

In [26]:

```python
plt.figure(figsize=(10, 6))

# Plot all metrics together
plt.plot(history.history['loss'], 'b-', label='Training Loss')
plt.plot(history.history['val_loss'], 'b--', label='Validation Loss')
plt.plot(history.history['accuracy'], 'r-', label='Training Accuracy')
plt.plot(history.history['val_accuracy'], 'r--', label='Validation Accuracy'

plt.title('Training History', fontsize=14)
plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Metric Value', fontsize=12)
plt.legend(fontsize=10, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)

# Add secondary y-axis for accuracy
plt.gca().twinx()
plt.ylabel('Accuracy', fontsize=12)
plt.ylim([0, 1])
```

```
plt.tight_layout()
plt.show()
```



Model Testing

- Taking 10 random sample from the test directory
- Calculating Accuracy and loss

In [27]:
```
# Load the best saved model
model.load_weights('best_model.h5')

# Function to test random images
def test_random_images(num_images=10):
    # Get all test image paths
    test_image_paths = []
    for class_name in class_names:
        class_dir = os.path.join(test_dir, class_name)
        if os.path.exists(class_dir):
            for img_name in os.listdir(class_dir):
                test_image_paths.append(os.path.join(class_dir, img_name))

    # Select random images
    selected_images = random.sample(test_image_paths, min(num_images, len(te

    plt.figure(figsize=(15, 10))
    for i, img_path in enumerate(selected_images):
        # Load and preprocess image
        img = Image.open(img_path)
        img = img.resize(img_size)
        img_array = np.array(img) / 255.0
        img_array = np.expand_dims(img_array, axis=0)
```

```
        # Get true label
        true_class = os.path.basename(os.path.dirname(img_path))

        # Predict
        preds = model.predict(img_array)
        pred_class = class_names[np.argmax(preds)]
        confidence = np.max(preds)

        # Plot
        plt.subplot(2, 5, i+1)
        plt.imshow(img)
        plt.title(f"True: {true_class}\nPred: {pred_class}\nConf: {confidenc
        plt.axis('off')

    plt.tight_layout()
    plt.show()

# Test with 10 random images
test_random_images(10)

# Evaluate on full test set if needed
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)

loss, accuracy = model.evaluate(test_generator)
print(f"\nTest Accuracy: {accuracy*100:.2f}%")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 196ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 187ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 190ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 181ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 185ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 196ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 193ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 200ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 191ms/step
```

```
Found 4170 images belonging to 55 classes.
131/131 ━━━━━━━━━━━━━━━━━━━━ 127s 964ms/step - accuracy: 0.9300 - loss: 0.29
78

Test Accuracy: 93.57%
```

Model Metrics

In [28]:

```python
def plot_confusion_matrix(y_true, y_pred, classes):
    from sklearn.metrics import confusion_matrix
    import seaborn as sns

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d',
                xticklabels=classes,
                yticklabels=classes,
                cmap='Blues')
    plt.title('Confusion Matrix')
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.xticks(rotation=45)
    plt.show()

def evaluate_model_metrics(model, test_generator, class_names):
    # Get predictions
    y_true = test_generator.classes
    y_pred_probs = model.predict(test_generator)
    y_pred = np.argmax(y_pred_probs, axis=1)

    # Calculate metrics
    precision = precision_score(y_true, y_pred, average='weighted')
    recall = recall_score(y_true, y_pred, average='weighted')
    f1 = f1_score(y_true, y_pred, average='weighted')

    # Print metrics
```

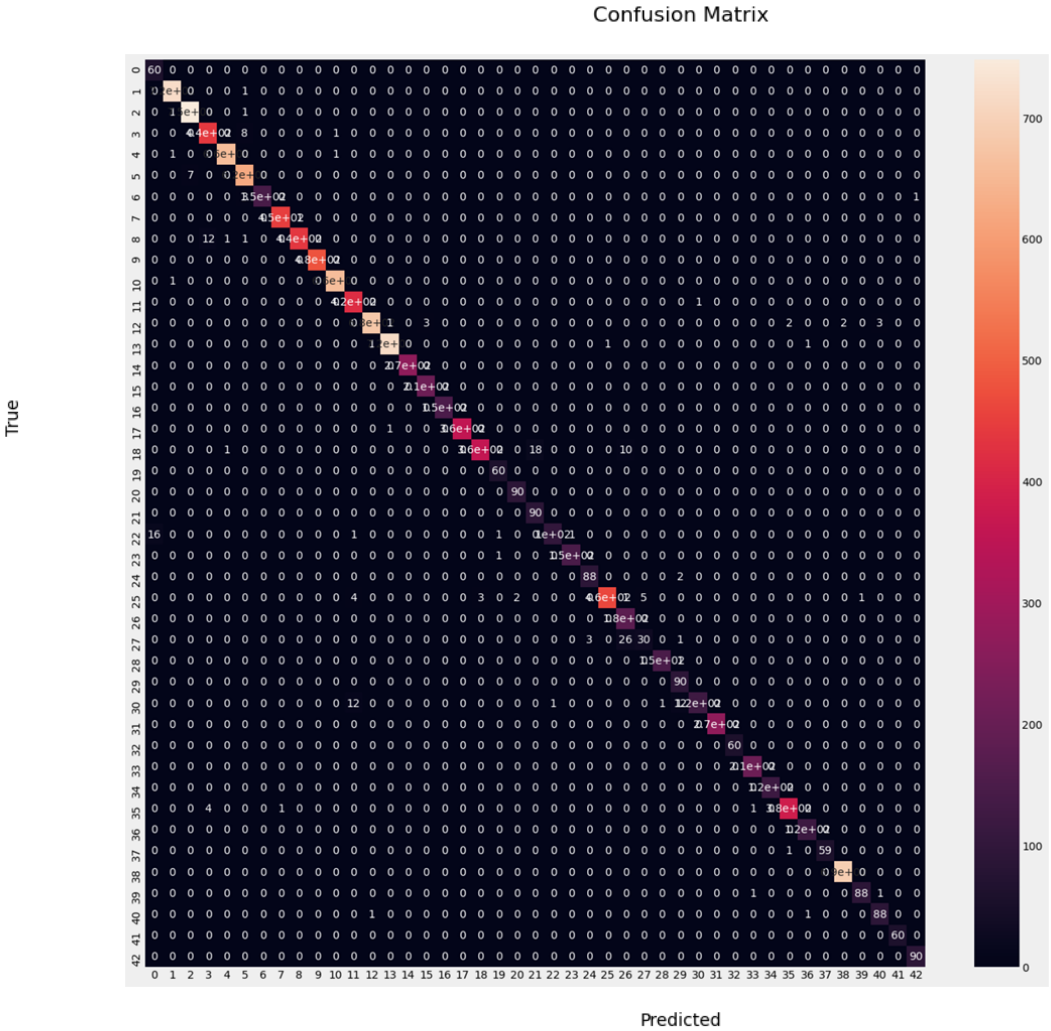
```
    print(f"{'Metric':<15}{'Value':>10}")
    print("-"*25)
    print(f"{'Precision':<15}{precision:>10.4f}")
    print(f"{'Recall':<15}{recall:>10.4f}")
    print(f"{'F1 Score':<15}{f1:>10.4f}")

    # Detailed report
    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, target_names=class_names))

    # Confusion matrix visualization
    plot_confusion_matrix(y_true, y_pred, class_names)

    return {'precision': precision, 'recall': recall, 'f1': f1}


def plot_roc_auc_curve(model, test_generator, class_names):
    # Get true labels and predictions
    y_true = test_generator.classes
    y_pred_probs = model.predict(test_generator)

    # Binarize the true labels for multi-class ROC
    y_true_bin = label_binarize(y_true, classes=np.arange(len(class_names)))

    # Compute ROC curve and ROC area for each class
    fpr = dict()
    tpr = dict()
    roc_auc = dict()

    for i in range(len(class_names)):
        fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_pred_probs[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Compute micro-average ROC curve and ROC area
    fpr["micro"], tpr["micro"], _ = roc_curve(y_true_bin.ravel(), y_pred_prc
    roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

    # Compute macro-average ROC curve and ROC area
    all_fpr = np.unique(np.concatenate([fpr[i] for i in range(len(class_name
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(len(class_names)):
        mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])
    mean_tpr /= len(class_names)
    fpr["macro"] = all_fpr
    tpr["macro"] = mean_tpr
    roc_auc["macro"] = auc(fpr["macro"], tpr["macro"])

    # Plot all ROC curves
    plt.figure(figsize=(10, 8))

    # Plot micro-average
    plt.plot(fpr["micro"], tpr["micro"],
             label=f'micro-average ROC (AUC = {roc_auc["micro"]:.2f})',
             color='deeppink', linestyle=':', linewidth=4)

    # Plot macro-average
```

```
    plt.plot(fpr["macro"], tpr["macro"],
             label=f'macro-average ROC (AUC = {roc_auc["macro"]:.2f})',
             color='navy', linestyle=':', linewidth=4)

    # Plot individual classes
    colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'green', 'red',
                    'purple', 'brown', 'pink', 'gray', 'olive'])
    for i, color in zip(range(len(class_names)), colors):
        plt.plot(fpr[i], tpr[i], color=color, lw=2,
                 label=f'Class {class_names[i]} (AUC = {roc_auc[i]:.2f})')

    plt.plot([0, 1], [0, 1], 'k--', lw=2)
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title('Multi-class ROC Curve')
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    return roc_auc

metrics = evaluate_model_metrics(model, test_generator, class_names)
roc_auc_scores = plot_roc_auc_curve(model, test_generator, class_names)
print("\nAUC Scores:")
for i, class_name in enumerate(class_names):
    print(f"{class_name}: {roc_auc_scores[i]:.4f}")
print(f"\nMicro-average AUC: {roc_auc_scores['micro']:.4f}")
print(f"Macro-average AUC: {roc_auc_scores['macro']:.4f}")
```

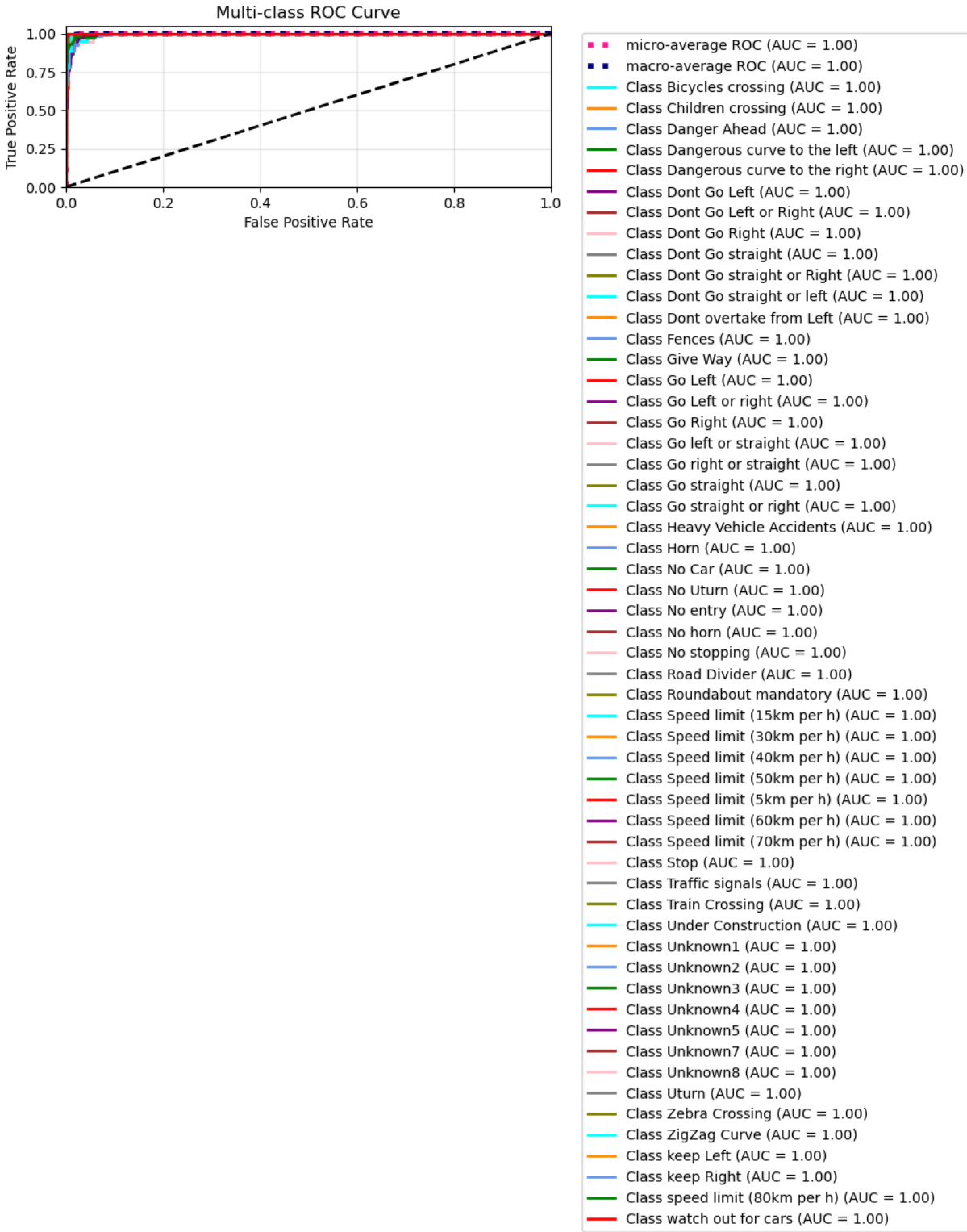
```
131/131 ━━━━━━━━━━━━━━━━━━━━ 134s 994ms/step
Metric              Value
-------------------------
Precision          0.9386
Recall             0.9357
F1 Score           0.9349

Classification Report:
                              precision    recall  f1-score   support

           Bicycles crossing       1.00      0.98      0.99       190
           Children crossing       0.83      0.86      0.85        58
                Danger Ahead       0.90      0.69      0.78        26
 Dangerous curve to the left       0.62      0.67      0.65        30
Dangerous curve to the right       0.67      0.59      0.62        34
                Dont Go Left       0.99      1.00      0.99       138
       Dont Go Left or Right       0.96      1.00      0.98        96
               Dont Go Right       0.94      0.83      0.88        36
            Dont Go straight       1.00      0.91      0.96        70
   Dont Go straight or Right       0.00      0.00      0.00         2
    Dont Go straight or left       1.00      0.50      0.67         8
     Dont overtake from Left       0.95      0.98      0.97       128
                      Fences       0.93      0.96      0.95        56
                   Give Way        0.00      0.00      0.00         2
                    Go Left        1.00      0.89      0.94        18
            Go Left or right       0.86      0.86      0.86        14
                   Go Right        1.00      0.98      0.99       100
         Go left or straight       0.41      0.47      0.44        30
        Go right or straight       0.73      0.80      0.77        82
                 Go straight       1.00      1.00      1.00        12
        Go straight or right       1.00      0.89      0.94        18
     Heavy Vehicle Accidents       0.75      0.75      0.75         8
                        Horn       1.00      1.00      1.00        44
                      No Car       1.00      0.97      0.99       142
                   No Uturn        1.00      1.00      1.00        22
                   No entry        1.00      1.00      1.00       162
                    No horn        0.94      0.98      0.96       130
                 No stopping       0.99      1.00      0.99       324
                Road Divider       1.00      0.57      0.73        14
        Roundabout mandatory       1.00      1.00      1.00        28
    Speed limit (15km per h)       0.78      0.90      0.84        40
    Speed limit (30km per h)       0.79      0.93      0.85        80
    Speed limit (40km per h)       0.84      0.98      0.90       268
    Speed limit (50km per h)       0.80      0.88      0.84       102
     Speed limit (5km per h)       0.95      0.97      0.96       118
    Speed limit (60km per h)       0.91      0.76      0.83       194
    Speed limit (70km per h)       0.97      0.92      0.95        78
                        Stop       1.00      1.00      1.00        36
             Traffic signals       1.00      1.00      1.00         4
              Train Crossing       0.83      0.83      0.83        12
          Under Construction       1.00      1.00      1.00        10
                   Unknown1        1.00      0.88      0.93        32
                   Unknown2        0.78      0.78      0.78        18
                   Unknown3        0.83      0.94      0.88        32
                   Unknown4        0.83      0.83      0.83        24
                   Unknown5        0.91      0.95      0.93        42
```

```
                   Unknown7       1.00      1.00      1.00       110
                   Unknown8       1.00      1.00      1.00         6
                      Uturn       1.00      1.00      1.00        42
             Zebra Crossing       0.94      0.97      0.96       156
               ZigZag Curve       1.00      0.56      0.71        18
                  keep Left       1.00      1.00      1.00         2
                 keep Right       1.00      1.00      1.00       126
    speed limit (80km per h)       1.00      0.79      0.88       152
          watch out for cars       1.00      1.00      1.00       446

                   accuracy                           0.94      4170
                  macro avg       0.88      0.85      0.86      4170
               weighted avg       0.94      0.94      0.93      4170
```

```
C:\Users\mohan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.
0 in labels with no predicted samples. Use `zero_division` parameter to cont
rol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\mohan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.
0 in labels with no predicted samples. Use `zero_division` parameter to cont
rol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\mohan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.
0 in labels with no predicted samples. Use `zero_division` parameter to cont
rol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\mohan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.p
y:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.
0 in labels with no predicted samples. Use `zero_division` parameter to cont
rol this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



131/131 ━━━━━━━━━━━━━━━━━━━━ **128s** 973ms/step


Multi-class ROC Curve
True Positive Rate
False Positive Rate
0.00
0.25
0.50
0.75
1.00
0.0
0.2
0.4
0.6
0.8
1.0
micro-average ROC (AUC = 1.00)
macro-average ROC (AUC = 1.00)
Class Bicycles crossing (AUC = 1.00)
Class Children crossing (AUC = 1.00)
Class Danger Ahead (AUC = 1.00)
Class Dangerous curve to the left (AUC = 1.00)
Class Dangerous curve to the right (AUC = 1.00)
Class Dont Go Left (AUC = 1.00)
Class Dont Go Left or Right (AUC = 1.00)
Class Dont Go Right (AUC = 1.00)
Class Dont Go straight (AUC = 1.00)
Class Dont Go straight or Right (AUC = 1.00)
Class Dont Go straight or left (AUC = 1.00)
Class Dont overtake from Left (AUC = 1.00)
Class Fences (AUC = 1.00)
Class Give Way (AUC = 1.00)
Class Go Left (AUC = 1.00)
Class Go Left or right (AUC = 1.00)
Class Go Right (AUC = 1.00)
Class Go left or straight (AUC = 1.00)
Class Go right or straight (AUC = 1.00)
Class Go straight (AUC = 1.00)
Class Go straight or right (AUC = 1.00)
Class Heavy Vehicle Accidents (AUC = 1.00)
Class Horn (AUC = 1.00)
Class No Car (AUC = 1.00)
Class No Uturn (AUC = 1.00)
Class No entry (AUC = 1.00)
Class No horn (AUC = 1.00)
Class No stopping (AUC = 1.00)
Class Road Divider (AUC = 1.00)
Class Roundabout mandatory (AUC = 1.00)
Class Speed limit (15km per h) (AUC = 1.00)
Class Speed limit (30km per h) (AUC = 1.00)
Class Speed limit (40km per h) (AUC = 1.00)
Class Speed limit (50km per h) (AUC = 1.00)
Class Speed limit (5km per h) (AUC = 1.00)
Class Speed limit (60km per h) (AUC = 1.00)
Class Speed limit (70km per h) (AUC = 1.00)
Class Stop (AUC = 1.00)
Class Traffic signals (AUC = 1.00)
Class Train Crossing (AUC = 1.00)
Class Under Construction (AUC = 1.00)
Class Unknown1 (AUC = 1.00)
Class Unknown2 (AUC = 1.00)
Class Unknown3 (AUC = 1.00)
Class Unknown4 (AUC = 1.00)
Class Unknown5 (AUC = 1.00)
Class Unknown7 (AUC = 1.00)
Class Unknown8 (AUC = 1.00)
Class Uturn (AUC = 1.00)
Class Zebra Crossing (AUC = 1.00)
Class ZigZag Curve (AUC = 1.00)
Class keep Left (AUC = 1.00)
Class keep Right (AUC = 1.00)
Class speed limit (80km per h) (AUC = 1.00)
Class watch out for cars (AUC = 1.00)

AUC Scores:
Bicycles crossing: 1.0000
Children crossing: 0.9991
Danger Ahead: 0.9979
Dangerous curve to the left: 0.9973
Dangerous curve to the right: 0.9973
Dont Go Left: 1.0000
Dont Go Left or Right: 1.0000
Dont Go Right: 0.9967
Dont Go straight: 0.9999
Dont Go straight or Right: 1.0000
Dont Go straight or left: 0.9995
Dont overtake from Left: 0.9992
Fences: 0.9999
Give Way: 1.0000
Go Left: 1.0000
Go Left or right: 0.9999
Go Right: 0.9990
Go left or straight: 0.9967
Go right or straight: 0.9972
Go straight: 1.0000
Go straight or right: 0.9998
Heavy Vehicle Accidents: 0.9999
Horn: 1.0000
No Car: 1.0000
No Uturn: 1.0000
No entry: 1.0000
No horn: 0.9999
No stopping: 1.0000
Road Divider: 0.9999
Roundabout mandatory: 1.0000
Speed limit (15km per h): 0.9973
Speed limit (30km per h): 0.9990
Speed limit (40km per h): 0.9978
Speed limit (50km per h): 0.9973
Speed limit (5km per h): 0.9998
Speed limit (60km per h): 0.9959
Speed limit (70km per h): 0.9997
Stop: 1.0000
Traffic signals: 1.0000
Train Crossing: 0.9996
Under Construction: 1.0000
Unknown1: 0.9998
Unknown2: 0.9977
Unknown3: 0.9998
Unknown4: 0.9995
Unknown5: 0.9998
Unknown7: 1.0000
Unknown8: 1.0000
Uturn: 1.0000
Zebra Crossing: 0.9995
ZigZag Curve: 0.9983
keep Left: 1.0000
keep Right: 1.0000
speed limit (80km per h): 0.9986
watch out for cars: 1.0000

```
Micro-average AUC: 0.9996
Macro-average AUC: 0.9992
```

Extracting Region Of Interests And Predicting

- Extract possible road signs from a car dashcam screencapture
- Pass the extracted ROI images from the ROIs directory to the pretrained model
- Show the predicted output with highest confidence score

Importing necessary libraries

In [29]:
```
import cv2
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import preprocess_input
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import math
```

ROI Extraction

In [30]:
```
# Remove a specific color from an existing mask
def exclude_color_from_mask(mask, image, exclude_hex):
    # Convert hex to BGR
    hex_value = exclude_hex.lstrip('#')
    bgr_color = np.array([int(hex_value[i:i+2], 16) for i in (4, 2, 0)])

    # Create exclusion mask
    exclude_mask = cv2.inRange(image, bgr_color - 20, bgr_color + 20)

    # Remove excluded color from original mask
    return cv2.bitwise_and(mask, cv2.bitwise_not(exclude_mask))


# Creates a mask for the image based on hsv color ranges
def preProcessFrame(img):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    color_ranges = {
        'red': [
            (np.array([0, 70, 50]), np.array([10, 255, 255])),
            (np.array([170, 70, 50]), np.array([180, 255, 255]))
        ],
        'blue': [  # Standard blue
            (np.array([100, 50, 50]), np.array([140, 255, 255]))
        ],
        'yellow': [
            (np.array([20, 100, 100]), np.array([30, 255, 255]))
        ]
    }
```

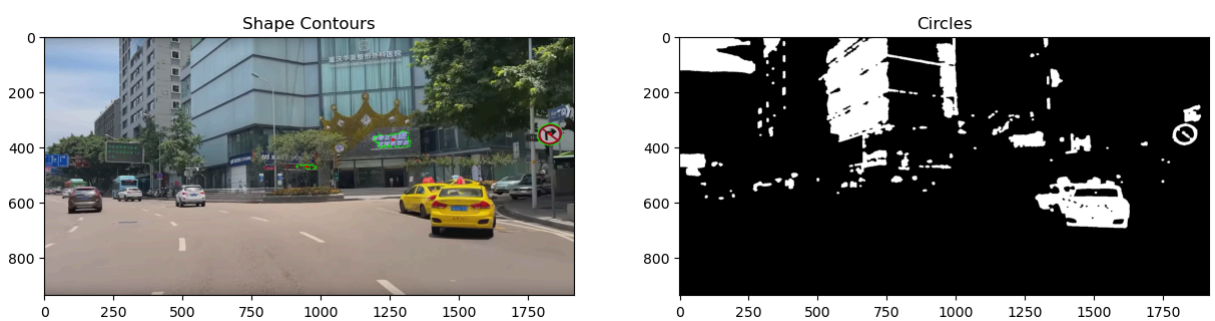
```
    mask = np.zeros_like(hsv[:,:,0])

    # Filter entities based on HSV color ranges
    for color, ranges in color_ranges.items():
        for lower, upper in ranges:
            mask = cv2.bitwise_or(mask, cv2.inRange(hsv, lower, upper))

        # Remove noise causing colors from the mask.
        mask = exclude_color_from_mask(mask, img, "#697787")
        mask = exclude_color_from_mask(mask, img, "#B8D3EA")

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=2)


        mask = cv2.GaussianBlur(mask, (1,1), 0.5)

    return mask


def detectCircularEntity(image, min_radius=23, max_radius=150):
    blurred = cv2.bilateralFilter(image, 9, 75, 75)

    # Detect edges with Canny (dynamic thresholds)
    v = np.median(blurred)
    lower = int(max(0, (1.0 - 0.33) * v))
    upper = int(min(255, (1.0 + 0.33) * v))
    edges = cv2.Canny(blurred, lower, upper)

    # connect broken edges
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
    closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)

    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPR

    output = image.copy()
    output = cv2.cvtColor(output, cv2.COLOR_GRAY2BGR)
    circles = []

    for cnt in contours:
        if cv2.contourArea(cnt) < 100:
            continue

        # Calculate circularity
        perimeter = cv2.arcLength(cnt, True)
        area = cv2.contourArea(cnt)
        if perimeter == 0:
            continue
        circularity = 4 * np.pi * area / (perimeter**2)

        # Check if contour is circular enough
        if 0.7 <= circularity <= 1.3:
            # Get enclosing circle
            (x, y), radius = cv2.minEnclosingCircle(cnt)
```

```
            center = (int(x), int(y))
            radius = int(radius)

            if min_radius <= radius <= max_radius:
                cv2.circle(output, center, radius, (0, 255, 0), 2)
                cv2.circle(output, center, 2, (0, 0, 255), 3)
                circles.append((center, radius))

    return output, circles

def findContours(img, min_area = 550, max_aspect = 1.5):
    edges = cv2.Canny(img, 10, 30)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPRO
    filtered_contours = []
    for cnt in contours:
        approx = cv2.approxPolyDP(cnt, 0.04 * cv2.arcLength(cnt, True), True

        if len(approx) in [3, 4, 8]: #Triangle , rectangle, octagon
            area = cv2.contourArea(cnt)
            if area < min_area:
                continue
            filtered_contours.append(cnt)

    return filtered_contours

def drawContours(cnts_list, img):
    imageWithBoxes = img.copy()
    try:
        imageWithBoxes = cv2.cvtColor(imageWithBoxes, cv2.COLOR_GRAY2BGR)
    except:
        pass
    for cnts in cnts_list:
        for cnt in cnts:
            x, y, w, h = cv2.boundingRect(cnt)
            cv2.rectangle(imageWithBoxes, (x, y), (x+w, y+h), (0,255,0), 2)

        # Calculate contour center.
        M = cv2.moments(cnts)
        if M["m00"] != 0:
            cX = int(M["m10"] / M["m00"])
            cY = int(M["m01"] / M["m00"])
        else:
            cX, cY = 0, 0

        cv2.circle(imageWithBoxes, (cX, cY), 5, (0, 0, 255), -1)

    return imageWithBoxes

def getMaxDistanceFromCenter(contour, center):
    points = contour.reshape(-1, 2)
    distances = np.sqrt((points[:, 0] - center[0])**2 + (points[:,1] - cente
    max_index = np.argmax(distances)
    max_distance = distances[max_index]

    return max_distance
```

```python
def getRegionOfInterests(cnts_list, img, extra_padding=35):
    rois = []
    for cnts in cnts_list:
        img_bak = img.copy()
        x,y,w,h = cv2.boundingRect(cnts)

        cropped = img_bak[max(0, int(y)-extra_padding):int(y+h)+extra_paddir
        if(len(cropped != 0)):
            rois.append(cropped)

        # for cnt in cnts:
        #     x, y, w, h = cv2.boundingRect(cnt)

        #     #detect center of the contours
        #     M = cv2.moments(cnts)
        #     if M["m00"] != 0:
        #         cX = int(M["m10"] / M["m00"])
        #         cY = int(M["m01"] / M["m00"])
        #     else:
        #         cX, cY = 0, 0

        #     max_distance_from_center = getMaxDistanceFromCenter(cnt, (cX,

        #     x_bound_min, x_bound_max = int(max(0, cX-max_distance_from_cer
        #     y_bound_min, y_bound_max = int(max(0, cY-max_distance_from_cer

        #     cropped = img[x_bound_min:x_bound_max, y_bound_min:y_bound_max
        #     if len(cropped) != 0:
        #         rois.append(cropped)



    return rois


def extractPossibleROIs(img, extra_padding=35) -> dict:

    preProcessedFrame = preProcessFrame(img)
    shapeContours = findContours(preProcessedFrame)
    annotated_circular_signs, circles_info = detectCircularEntity(preProcess

    annotated_shape_frame = drawContours(shapeContours, img)
    circular_entities = []
    for circle in circles_info:
        y_bound_min, y_bound_max = int(max(0, circle[0][0]-circle[1]-extra_p
        x_bound_min, x_bound_max = int(max(0, circle[0][1]-circle[1]-extra_p
        cropped = img[x_bound_min:x_bound_max, y_bound_min:y_bound_max]
        circular_entities.append(cropped)
    rois = getRegionOfInterests(shapeContours, img, extra_padding)

    return {
        "annotated_circular_signs" : annotated_circular_signs,
        "circular_entities" : circular_entities,
        "rois" : rois,
        "pre_processed_frame" : preProcessedFrame,
        "annotated_shape_frame" : annotated_shape_frame
```

```
    }

if not os.path.isdir("ROIs"):
  os.mkdir("ROIs")

input_image = "car_dashcam_footage.png"

image = cv2.imread(input_image)

input_image = input_image.split(".")[0]

possibleROIs = extractPossibleROIs(image)
cv2.imwrite(f"{input_image}_cirles.png", possibleROIs["annotated_circular_si
cv2.imwrite(f"{input_image}_contours.png", possibleROIs["annotated_shape_fra
cv2.imwrite(f"{input_image}_preProcessedFrame.png", possibleROIs["pre_proces
for i, circular_entity in enumerate(possibleROIs["circular_entities"]):
    cv2.imwrite(f"ROIs/{input_image}_circularEntity_{i}.png", circular_entit

for i, roi in enumerate(possibleROIs["rois"]):
    cv2.imwrite(f"ROIs/{input_image}_roi_{i}.png", roi)

plt.rcParams["figure.figsize"] = (15,10)
f, axarr = plt.subplots(1,2)

annotated_shape_image, annotated_circles_image = Image.open(f"{input_image}_

axarr[1].imshow(annotated_circles_image)
axarr[1].set_title("Circles")
axarr[0].imshow(annotated_shape_image)
axarr[0].set_title("Shape Contours")
```

Out[30]: Text(0.5, 1.0, 'Shape Contours')



Predicting

- Show images whose confidence score is atleast 60%

In [32]:
```
def predict_and_display_high_confidence_signs(directory, confidence_threshol
    image_paths = [os.path.join(directory, f) for f in os.listdir(directory)
                   if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    if not image_paths:
        print("No images found in directory!")
        return
```

```
    high_confidence_predictions = []

    for img_path in image_paths:
        # Load and preprocess image
        img = Image.open(img_path).resize(img_size)
        img_array = img_to_array(img)
        img_array = np.expand_dims(img_array, axis=0)
        img_array = preprocess_input(img_array)

        # Make prediction
        preds = model.predict(img_array)
        top_class = class_names[np.argmax(preds)]
        top_confidence = np.max(preds)

        if top_confidence >= confidence_threshold:
            high_confidence_predictions.append({
                'path': img_path,
                'class': top_class,
                'confidence': top_confidence,
                'image': img
            })

    if not high_confidence_predictions:
        print(f"No predictions met the {confidence_threshold:.0%} confidence
        return

    # Sort predictions by confidence (highest first)
    high_confidence_predictions.sort(key=lambda x: x['confidence'], reverse=

    # Calculate grid size for display
    num_images = len(high_confidence_predictions)
    cols = 4  # Number of columns in the grid
    rows = math.ceil(num_images / cols)

    plt.figure(figsize=(cols * 4, rows * 4))

    for i, pred in enumerate(high_confidence_predictions, 1):
        plt.subplot(rows, cols, i)
        plt.imshow(pred['image'])
        plt.title(f"{pred['class']}\nConfidence: {pred['confidence']:.2%}")
        plt.axis('off')

    plt.tight_layout()
    plt.show()

    print(f"\nFound {len(high_confidence_predictions)} images with confidenc
    for pred in high_confidence_predictions:
        print(f"- {os.path.basename(pred['path'])}: {pred['class']} ({pred['

predict_and_display_high_confidence_signs("ROIs", confidence_threshold=0.45)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 191ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 190ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 191ms/step
```

```
Found 1 images with confidence ≥60%:
- car_dashcam_footage_roi_2.png: Dont Go straight (75.70%)
```